

Informe de cambios MVP - Servify

Fecha: 14 de mayo de 2026

Proyecto: Servify

Backend: Java 21 + Spring Boot + Maven

Arquitectura: monolito modular en capas

Persistencia prevista: PostgreSQL

Cliente previsto: aplicacion movil React Native consumiendo API REST JSON

1. Objetivo del cambio

Se agregaron clases faltantes del backend para completar el esqueleto del MVP en los modulos `solicitudes`, `autenticacion` y `administracion`, respetando la arquitectura modular existente.

El objetivo no fue implementar negocio real, sino dejar contratos, DTOs, puertos y servicios preparados para que en una siguiente etapa se pueda completar la logica sin romper la separacion entre modulos.

Las clases agregadas siguen estas reglas:

- Los puertos `in` y `out` son interfaces.
- Los services son clases concretas.
- Cada service implementa su use case correspondiente.
- Los DTOs de application son clases simples de datos.
- No se usan `record`.
- No se agrego API REST ni infraestructura.
- No se agrego logica real de negocio.
- Los metodos pendientes tienen `TODO` y lanzan `UnsupportedOperationException`.
- La comunicacion entre modulos se hace con `UUID` y puertos cross-module.
- No se exponen entidades de dominio de un modulo dentro de otro modulo.
- `shared` se mantiene como lugar para conceptos transversales estables.

2. Estado arquitectonico respetado

La solucion esta organizada como monolito modular. Cada modulo mantiene sus capas:

- `domain`: modelo, enumtypes y servicios de dominio propios del modulo.
- `application.dto`: comandos y resultados usados por casos de uso.
- `application.port.in`: casos de uso expuestos por el modulo.
- `application.port.out`: puertos requeridos para persistencia o colaboracion externa.
- `application.service`: orquestadores de casos de uso.

La regla central preservada fue no depender de implementaciones concretas ni de entidades de dominio externas. Cuando un modulo necesita algo de otro, se declaro un puerto con datos simples, principalmente `UUID`, `String`, tipos primitivos, `BigDecimal`, fechas y value objects de `shared`.

3. Dependencias colgantes detectadas y resueltas con puertos

Antes de generar las clases se identificaron las siguientes dependencias pendientes:

1. `solicitudes` necesita consultar publicaciones compatibles.
 - Se resolvió con `PublicacionesCompatiblesPort`.
 - No importa DTOs ni entidades de publicaciones.
 - Devuelve un `Map<UUID, UUID>`; donde la clave representa la publicación compatible y el valor el prestador asociado.
2. `solicitudes` necesita parámetros de configuración de distribución.
 - Se resolvió con `ConfiguracionDistribucionPort`.
 - Expone radios, tiempos de espera y precio mínimo de referencia.
3. `autenticacion` necesita validar usuarios autenticables.
 - Se resolvió con `UsuarioAutenticablePort`.
 - No importa `Usuario` ni `EstadoUsuario`.
4. `administracion` necesita operar sobre usuarios administrables.
 - Se resolvió con `UsuarioAdministrablePort`.
 - Trabaja con `UUID` y `TipoMedida` del propio módulo `administracion`.
5. `administracion` necesita moderar publicaciones.
 - Se resolvió con `PublicacionModerablePort`.
 - No importa `PublicacionServicio` ni `EstadoPublicacion`.
 - El estado destino viaja como `String` para evitar acoplar `administracion` al dominio de publicaciones.
6. No se duplicó gestión de categorías en `administracion`.
 - La `administracion` no agrega clases de categoría porque ese comportamiento pertenece al módulo `publicaciones`.

4. Cambios agregados en módulo solicitudes

4.1 DTOs agregados

Ruta base: `src/main/java/com/servify/solicitudes/application/dto`

Clases agregadas:

- `SolicitudRecibidaResult`
- `EstadoAsignacionSolicitudResult`
- `ConfirmarAsignacionSolicitudCommand`
- `ResolverContraofertaCommand`
- `TipoDecisionSolicitud`
- `ContraofertaResult`
- `AsignacionServicioResult`

Detalle:

SolicitudRecibidaResult

- Representa la vista de una solicitud recibida por un prestador.
- Incluye ids de solicitud, distribucion, publicacion, prestador, solicitante y categoria.
- Usa ModalidadServicio, UbicacionSolicitudResult y DisponibilidadHorariaResult.
- Incluye estado de distribucion, ronda, fecha de envio y fecha de expiracion.
- Usa builder para evitar constructores grandes.

EstadoAsignacionSolicitudResult

- Representa el estado de asignacion visible para una solicitud.
- Incluye solicitudId, solicitanteId, EstadoSolicitud, asignacion, contraofertas pendientes y cantidad de distribuciones activas.
- Usa builder por ser un result compuesto.

ConfirmarAsignacionSolicitudCommand

- Comando para confirmar una asignacion desde una distribucion.
- Campos:
 - solicitudId
 - distribucionSolicitudId
 - solicitanteId

ResolverContraofertaCommand

- Comando para aceptar o rechazar una contraoferta.
- Campos:
 - contraofertaId
 - solicitanteId
 - decision

TipoDecisionSolicitud

- Enum simple para decisiones del solicitante:
 - ACEPTAR
 - RECHAZAR

ContraofertaResult

- Resultado de lectura de una contraoferta.
- Refleja datos propios del dominio Contraoferta.
- Usa EstadoContraoferta.
- Usa builder.

AsignacionServicioResult

- Resultado de lectura de una asignacion.

- Refleja datos propios del dominio `AsignacionServicio`.
- Usa `EstadoAsignacion`.
- Usa builder.

4.2 Puertos de entrada agregados

Ruta base: `src/main/java/com/servify/solicitudes/application/port/in`

Interfaces agregadas:

- `ConfirmarAsignacionSolicitudUseCase`
- `ResolverContraofertaUseCase`
- `ObtenerEstadoAsignacionSolicitudUseCase`

Contratos:

- `ConfirmarAsignacionSolicitudUseCase.confirmar(...)` devuelve `AsignacionServicioResult`.
- `ResolverContraofertaUseCase.resolver(...)` devuelve `ContraofertaResult`.
- `ObtenerEstadoAsignacionSolicitudUseCase.obtenerEstado(...)` devuelve `EstadoAsignacionSolicitudResult`.

4.3 Puertos de salida agregados

Ruta base: `src/main/java/com/servify/solicitudes/application/port/out`

Interfaces agregadas:

- `PublicacionesCompatiblesPort`
- `ConfiguracionDistribucionPort`

`PublicacionesCompatiblesPort`

- Puerto cross-module hacia publicaciones.
- No importa clases de publicaciones.
- Trabaja con:
 - `UUID`
 - `BigDecimal`
 - `Map`
 - `ModalidadServicio`
 - `Ubicacion`
 - `DisponibilidadHoraria`

`ConfiguracionDistribucionPort`

- Puerto para obtener parametros de configuracion necesarios para distribuir solicitudes.
- Expone:
 - radio inicial
 - radio de expansion

- tiempo de espera
- precio minimo de referencia

4.4 Services agregados

Ruta base: `src/main/java/com/servify/solicitudes/application/service`

Clases agregadas:

- `ConfirmarAsignacionSolicitudService`
- `ResolverContraofertaService`
- `ObtenerEstadoAsignacionSolicitudService`

`ConfirmarAsignacionSolicitudService`

- Implementa `ConfirmarAsignacionSolicitudUseCase`.
- Depende de:
 - `SolicitudServicioRepositoryPort`
 - `DistribucionSolicitudRepositoryPort`
 - `AsignacionServicioRepositoryPort`
 - `PoliticaAsignacionUnica`
- Deja preparado el flujo:
 1. Obtener solicitud.
 2. Obtener distribucion.
 3. Validar solicitante.
 4. Validar que la distribucion pueda confirmarse.
 5. Construir asignacion.
 6. Cerrar distribuciones restantes.
 7. Devolver result.
- Todos los pasos estan pendientes con `TODO` y `UnsupportedOperationException`.

`ResolverContraofertaService`

- Implementa `ResolverContraofertaUseCase`.
- Depende de:
 - `ContraofertaRepositoryPort`
 - `DistribucionSolicitudRepositoryPort`
 - `SolicitudServicioRepositoryPort`
- Deja preparado el flujo:
 1. Obtener contraoferta.
 2. Obtener distribucion.
 3. Obtener solicitud.
 4. Validar que el solicitante pueda resolver.
 5. Aplicar decision.

6. Construir result.

ObtenerEstadoAsignacionSolicitudService

- Implementa `ObtenerEstadoAsignacionSolicitudUseCase`.
- Depende de:
 - `SolicitudServicioRepositoryPort`
 - `DistribucionSolicitudRepositoryPort`
 - `AsignacionServicioRepositoryPort`
 - `ContraofertaRepositoryPort`
- Deja preparado el armado de una vista agregada del estado de asignacion sin exponer entidades.

5. Cambios agregados en modulo autenticacion

5.1 Modelo de dominio agregado

Ruta base: `src/main/java/com/servify/autenticacion/domain/model`

Clase agregada:

- `RefreshToken`

RefreshToken

- Extiende `BaseEntity`.
- Representa un refresh token persistible del modulo autenticacion.
- Campos:
 - `usuarioId`
 - `credencialAccesoId`
 - `tokenHash`
 - `fechaEmision`
 - `fechaExpiracion`
 - `fechaRevocacion`
 - `activo`
- Incluye metodos de dominio pendientes:
 - `estaActivo`
 - `estaExpirado`
 - `fueRevocado`
 - `perteneceAUsuario`
 - `revocar`
- Los metodos no implementan negocio real todavia.

5.2 DTOs agregados

Ruta base: `src/main/java/com/servify/autenticacion/application/dto`

Clases agregadas:

- RegistrarCredencialesCommand
- IniciarSesionCommand
- CerrarSesionCommand
- RenovarTokenCommand
- SesionResult
- TokenResult

Detalle:

RegistrarCredencialesCommand

- Datos para registrar credenciales de acceso.
- Campos:
- `usuarioId`
- `emailAcceso`
- `passwordPlano`

IniciarSesionCommand

- Datos de login.
- Campos:
- `emailAcceso`
- `passwordPlano`

CerrarSesionCommand

- Datos para cerrar sesion.
- Campos:
- `usuarioId`
- `refreshToken`

RenovarTokenCommand

- Datos para renovar sesion.
- Campo:
- `refreshToken`

TokenResult

- Resultado simple para representar access token o refresh token.
- Campos:
- `token`
- `tipoToken`
- `fechaEmision`
- `fechaExpiracion`

SesionResult

- Resultado compuesto de una sesion iniciada o renovada.
- Campos:
- `usuarioId`
- `emailAcceso`
- `accessToken`
- `refreshToken`
- `fechaInicioSesion`
- Usa builder para evitar constructores grandes.

5.3 Puertos de entrada agregados

Ruta base: `src/main/java/com/servify/autenticacion/application/port/in`

Interfaces agregadas:

- `RegistrarCredencialesUseCase`
- `IniciarSesionUseCase`
- `CerrarSesionUseCase`
- `RenovarTokenUseCase`

Contratos:

- `RegistrarCredencialesUseCase.registrar(...)`
- `IniciarSesionUseCase.iniciar(...)`
- `CerrarSesionUseCase.cerrar(...)`
- `RenovarTokenUseCase.renovar(...)`

5.4 Puertos de salida agregados

Ruta base: `src/main/java/com/servify/autenticacion/application/port/out`

Interfaces agregadas:

- `CredencialAccesoRepositoryPort`
- `PasswordHasherPort`
- `TokenProviderPort`
- `RefreshTokenRepositoryPort`
- `UsuarioAutenticablePort`

Detalle:

`CredencialAccesoRepositoryPort`

- Puerto de persistencia de `CredencialAcceso`, entidad propia del modulo autenticacion.
- Metodos:
- `guardar`

- buscarPorId
- buscarPorUsuarioId
- buscarPorEmailAcceso
- existePorEmailAcceso

PasswordHasherPort

- Puerto para hasheo y comparacion de passwords.
- Permite que la implementacion concreta se ubique en infraestructura.

TokenProviderPort

- Puerto para emitir tokens y obtener hash de token.
- Devuelve TokenResult.

RefreshTokenRepositoryPort

- Puerto de persistencia de RefreshToken.
- Metodos:
- guardar
- buscarPorId
- buscarPorTokenHash
- buscarActivosPorUsuarioId

UsuarioAutenticablePort

- Puerto cross-module hacia usuarios.
- No importa entidades de usuarios.
- Metodos:
- existeUsuario
- puedeAutenticarse
- coincideEmailPrincipal

5.5 Services agregados

Ruta base: src/main/java/com/servify/autenticacion/application/service

Clases agregadas:

- RegistrarCredencialesService
- IniciarSesionService
- CerrarSesionService
- RenovarTokenService

RegistrarCredencialesService

- Implementa RegistrarCredencialesUseCase.
- Depende de:

- CredencialAccesoRepositoryPort
- PasswordHasherPort
- UsuarioAutenticablePort
- Deja preparado el flujo de validacion de usuario, construccion de credencial y guardado.

IniciarSesionService

- Implementa IniciarSesionUseCase.
- Depende de:
- CredencialAccesoRepositoryPort
- PasswordHasherPort
- TokenProviderPort
- RefreshTokenRepositoryPort
- UsuarioAutenticablePort
- Deja preparado el flujo de login, emision de access token, emision de refresh token y armado de SesionResult.

CerrarSesionService

- Implementa CerrarSesionUseCase.
- Depende de:
- RefreshTokenRepositoryPort
- TokenProviderPort
- Deja preparado el flujo de busqueda, validacion y revocacion de refresh token.

RenovarTokenService

- Implementa RenovarTokenUseCase.
- Depende de:
- RefreshTokenRepositoryPort
- CredencialAccesoRepositoryPort
- TokenProviderPort
- UsuarioAutenticablePort
- Deja preparado el flujo de validacion de refresh token, emision de nuevos tokens y armado de sesion.

6. Cambios agregados en modulo administracion

6.1 DTOs agregados

Ruta base: src/main/java/com/servify/administracion/application/dto

Clases agregadas:

- AplicarMedidaAdministrativaUsuarioCommand
- MedidaAdministrativaUsuarioResult
- ActualizarConfiguracionGeneralCommand

- ConfiguracionGeneralResult
- ModerarPublicacionCommand

Detalle:

AplicarMedidaAdministrativaUsuarioCommand

- Datos para aplicar una medida administrativa sobre un usuario.
- Campos:
 - `usuarioId`
 - `administradorId`
 - `tipoMedida`
 - `motivo`
 - `fechaFinVigencia`

MedidaAdministrativaUsuarioResult

- Resultado de lectura de una medida administrativa.
- Usa `TipoMedida`, propio del modulo administracion.
- Usa builder.

ActualizarConfiguracionGeneralCommand

- Datos para actualizar configuracion general.
- Campos:
 - `configuracionGeneralId`
 - `administradorId`
 - `radioBusquedaInicialKm`
 - `radioBusquedaExpansionKm`
 - `tiempoEsperaExpansionMinutos`
 - `validacionIdentidadRequerida`
 - `precioBaseMinimoReferencia`
 - `plataformaActiva`

ConfiguracionGeneralResult

- Resultado de lectura de configuracion general.
- Usa builder.

ModerarPublicacionCommand

- Comando para moderar publicacion desde administracion.
- Campos:
 - `publicacionServicioId`
 - `administradorId`
 - `estadoDestino`
 - `motivo`

- estadoDestino se dejo como String para no acoplar administracion al enum EstadoPublicacion de publicaciones.

6.2 Puertos de entrada agregados

Ruta base: src/main/java/com/servify/administracion/application/port/in

Interfaces agregadas:

- AplicarMedidaAdministrativaUsuarioUseCase
- ObtenerMedidasAdministrativasDeUsuarioUseCase
- ActualizarConfiguracionGeneralUseCase
- ObtenerConfiguracionGeneralUseCase
- ModerarPublicacionUseCase

Contratos:

- AplicarMedidaAdministrativaUsuarioUseCase.aplicar(...)
- ObtenerMedidasAdministrativasDeUsuarioUseCase.obtenerPorUsuarioId(...)
- ActualizarConfiguracionGeneralUseCase.actualizar(...)
- ObtenerConfiguracionGeneralUseCase.obtenerVigente()
- ObtenerConfiguracionGeneralUseCase.obtenerPorId(...)
- ModerarPublicacionUseCase.moderar(...)

6.3 Puertos de salida agregados

Ruta base: src/main/java/com/servify/administracion/application/port/out

Interfaces agregadas:

- MedidaAdministrativaUsuarioRepositoryPort
- ConfiguracionGeneralRepositoryPort
- UsuarioAdministrablePort
- PublicacionModerablePort

Detalle:

MedidaAdministrativaUsuarioRepositoryPort

- Puerto de persistencia de MedidaAdministrativaUsuario, entidad propia de administracion.
- Metodos:
 - guardar
 - buscarPorId
 - buscarPorUsuarioId
 - buscarActivasPorUsuarioId

ConfiguracionGeneralRepositoryPort

- Puerto de persistencia de ConfiguracionGeneral.

- Metodos:
- guardar
- buscarPorId
- obtenerVigente

UsuarioAdministrablePort

- Puerto cross-module hacia usuarios.
- No importa entidades de usuarios.
- Metodos:
- existeUsuario
- esAdministrador
- aplicarMedida

PublicacionModerablePort

- Puerto cross-module hacia publicaciones.
- No importa entidades ni enums de publicaciones.
- Metodos:
- existePublicacion
- moderarPublicacion

6.4 Services agregados

Ruta base: src/main/java/com/servify/administracion/application/service

Clases agregadas:

- AplicarMedidaAdministrativaUsuarioService
- ObtenerMedidasAdministrativasDeUsuarioService
- ActualizarConfiguracionGeneralService
- ObtenerConfiguracionGeneralService
- ModerarPublicacionService

AplicarMedidaAdministrativaUsuarioService

- Implementa AplicarMedidaAdministrativaUsuarioUseCase.
- Depende de:
- MedidaAdministrativaUsuarioRepositoryPort
- UsuarioAdministrablePort
- Deja preparado el flujo de validacion de administrador, validacion de usuario afectado, construccion de medida, guardado y armado de result.

ObtenerMedidasAdministrativasDeUsuarioService

- Implementa ObtenerMedidasAdministrativasDeUsuarioUseCase.
- Depende de:

- MedidaAdministrativaUsuarioRepositoryPort
- Deja preparado el listado de medidas por usuario.

ActualizarConfiguracionGeneralService

- Implementa ActualizarConfiguracionGeneralUseCase.
- Depende de:
- ConfiguracionGeneralRepositoryPort
- UsuarioAdministrablePort
- Deja preparado el flujo de validacion de administrador, obtencion de configuracion, aplicacion de cambios y armado de result.

ObtenerConfiguracionGeneralService

- Implementa ObtenerConfiguracionGeneralUseCase.
- Depende de:
- ConfiguracionGeneralRepositoryPort
- Deja preparado el acceso a configuracion vigente o por id.

ModerarPublicacionService

- Implementa ModerarPublicacionUseCase.
- Depende de:
- PublicacionModerablePort
- UsuarioAdministrablePort
- Deja preparado el flujo de validacion de administrador, validacion de publicacion y delegacion de moderacion al puerto.

7. Decisiones de diseno importantes

7.1 Builders en results grandes

Los resultados grandes o compuestos usan builder:

- SolicitudRecibidaResult
- EstadoAsignacionSolicitudResult
- ContraofertaResult
- AsignacionServicioResult
- SesionResult
- MedidaAdministrativaUsuarioResult
- ConfiguracionGeneralResult

Motivo:

- Evitar constructores gigantes.
- Mejorar legibilidad.
- Hacer mas claro el armado progresivo desde services.

- Mantener DTOs simples sin introducir frameworks externos.

7.2 Commands simples con constructores y getters

Los commands se dejaron como clases simples de datos con:

- constructor vacio
- constructor completo
- getters

Esto facilita serializacion/deserializacion futura desde API REST sin acoplar todavia a controllers.

7.3 Estado de publicacion en administracion como String

`ModerarPublicacionCommand` usa `String` `estadoDestino` y `PublicacionModerablePort` tambien recibe `String`.

Motivo:

- Evitar que `administracion` importe `EstadoPublicacion` desde `publicaciones.domain`.
- Respetar la regla de no exponer entidades o tipos de dominio de un modulo dentro de otro.
- La traduccion a enum real debe ocurrir en el adaptador de infraestructura o en el modulo publicaciones cuando implemente el puerto.

7.4 Puertos cross-module con UUID

Los puertos que cruzan modulos usan `UUID` para referenciar usuarios, publicaciones, solicitudes o credenciales.

Esto mantiene bajo acoplamiento y evita que un modulo dependa de agregados ajenos.

7.5 Sin logica real por ahora

Aunque los services ya dibujan los pasos esperados, no resuelven reglas reales todavia.

Cada metodo pendiente tiene un `TODO` y lanza `UnsupportedOperationException`, siguiendo la condicion solicitada.

8. Validaciones realizadas

Se ejecuto compilacion Maven con Java 21:

```
mvn compile
BUILD SUCCESS
Compiling 183 source files with javac release 21
```

Tambien se reviso estaticamente:

- No hay usos de `record` en las clases agregadas.
- No hay imports directos desde `solicitudes`, `autenticacion` o `administracion` hacia `usuarios` o `publicaciones`.
- Los puertos de entrada y salida son interfaces.
- Los services agregados implementan sus respectivos use cases.
- `usuarios` y `publicaciones` no fueron modificados en esta tanda.

9. Resultado de tests

Se ejecuto:

```
mvn test
```

Resultado:

- La compilacion de main y test pasa.
- Falla el test `com.servify.SerfivyApplicationTests`.
- El error no esta relacionado con las clases agregadas.

Error relevante:

```
Unable to find a @SpringBootApplication by searching packages upwards from the test.
```

Interpretacion:

Spring Boot no encuentra una clase principal anotada con `@SpringBootApplication` o `@SpringBootConfiguration` en el paquete esperado. Para resolverlo, en una etapa separada habria que crear o corregir la clase principal de la aplicacion, o ajustar el test con `@SpringBootTest(classes = ...)`.

No se modifico esto porque el pedido actual era generar clases faltantes de application/domain sin tocar API ni configuracion general.

10. Observacion sobre Maven Wrapper

En esta maquina, `mvnw.cmd` fallo antes de ejecutar Maven por un problema del script wrapper con PowerShell:

```
No se puede indizar en una matriz nula.  
Cannot start maven from wrapper
```

Para verificar el proyecto se uso directamente la distribucion Maven ya descargada por el wrapper en:

```
C:\Users\juanc\.m2\wrapper\dists\apache-maven-3.9.14-bin\...\apache-maven-3.9.14\bin\mvn.cmd
```

Esto no requirio modificar el wrapper del repositorio.

11. Archivos agregados

solicitudes

```
src/main/java/com/servify/solicitudes/application/dto/SolicitudRecibidaResult.java  
src/main/java/com/servify/solicitudes/application/dto/EstadoAsignacionSolicitudResult.java  
src/main/java/com/servify/solicitudes/application/dto/ConfirmarAsignacionSolicitudCommand.java  
src/main/java/com/servify/solicitudes/application/dto/ResolverContraofertaCommand.java  
src/main/java/com/servify/solicitudes/application/dto/TipoDecisionSolicitud.java  
src/main/java/com/servify/solicitudes/application/dto/ContraofertaResult.java  
src/main/java/com/servify/solicitudes/application/dto/AsignacionServicioResult.java  
src/main/java/com/servify/solicitudes/application/port/in/ConfirmarAsignacionSolicitudUseCase.java  
src/main/java/com/servify/solicitudes/application/port/in/ResolverContraofertaUseCase.java  
src/main/java/com/servify/solicitudes/application/port/in/ObtenerEstadoAsignacionSolicitudUseCase.java  
src/main/java/com/servify/solicitudes/application/port/out/PublicacionesCompatiblesPort.java  
src/main/java/com/servify/solicitudes/application/port/out/ConfiguracionDistribucionPort.java  
src/main/java/com/servify/solicitudes/application/service/ConfirmarAsignacionSolicitudService.java  
src/main/java/com/servify/solicitudes/application/service/ResolverContraofertaService.java  
src/main/java/com/servify/solicitudes/application/service/ObtenerEstadoAsignacionSolicitudService.java
```


autenticacion

```
src/main/java/com/servify/autenticacion/application/dto/RegistrarCredencialesCommand.java
src/main/java/com/servify/autenticacion/application/dto/IniciarSesionCommand.java
src/main/java/com/servify/autenticacion/application/dto/CerrarSesionCommand.java
src/main/java/com/servify/autenticacion/application/dto/RenovarTokenCommand.java
src/main/java/com/servify/autenticacion/application/dto/SesionResult.java
src/main/java/com/servify/autenticacion/application/dto/TokenResult.java
src/main/java/com/servify/autenticacion/application/port/in/RegistrarCredencialesUseCase.java
src/main/java/com/servify/autenticacion/application/port/in/IniciarSesionUseCase.java
src/main/java/com/servify/autenticacion/application/port/in/CerrarSesionUseCase.java
src/main/java/com/servify/autenticacion/application/port/in/RenovarTokenUseCase.java
src/main/java/com/servify/autenticacion/application/port/out/CredencialAccesoRepositoryPort.java
src/main/java/com/servify/autenticacion/application/port/out/PasswordHasherPort.java
src/main/java/com/servify/autenticacion/application/port/out/TokenProviderPort.java
src/main/java/com/servify/autenticacion/application/port/out/RefreshTokenRepositoryPort.java
src/main/java/com/servify/autenticacion/application/port/out/UsuarioAutenticablePort.java
src/main/java/com/servify/autenticacion/application/service/RegistrarCredencialesService.java
src/main/java/com/servify/autenticacion/application/service/IniciarSesionService.java
src/main/java/com/servify/autenticacion/application/service/CerrarSesionService.java
src/main/java/com/servify/autenticacion/application/service/RenovarTokenService.java
src/main/java/com/servify/autenticacion/domain/model/RefreshToken.java
```

administracion

```
src/main/java/com/servify/administracion/application/dto/AplicarMedidaAdministrativaUsuarioCommand.java
src/main/java/com/servify/administracion/application/dto/MedidaAdministrativaUsuarioResult.java
src/main/java/com/servify/administracion/application/dto/ActualizarConfiguracionGeneralCommand.java
src/main/java/com/servify/administracion/application/dto/ConfiguracionGeneralResult.java
src/main/java/com/servify/administracion/application/dto/ModerarPublicacionCommand.java
src/main/java/com/servify/administracion/application/port/in/AplicarMedidaAdministrativaUsuarioUseCase.java
src/main/java/com/servify/administracion/application/port/in/ObtenerMedidasAdministrativasDeUsuarioUseCase.java
src/main/java/com/servify/administracion/application/port/in/ActualizarConfiguracionGeneralUseCase.java
src/main/java/com/servify/administracion/application/port/in/ObtenerConfiguracionGeneralUseCase.java
src/main/java/com/servify/administracion/application/port/in/ModerarPublicacionUseCase.java
src/main/java/com/servify/administracion/application/port/out/MedidaAdministrativaUsuarioRepositoryPort.java
src/main/java/com/servify/administracion/application/port/out/ConfiguracionGeneralRepositoryPort.java
src/main/java/com/servify/administracion/application/port/out/UsuarioAdministrablePort.java
src/main/java/com/servify/administracion/application/port/out/PublicacionModerablePort.java
src/main/java/com/servify/administracion/application/service/AplicarMedidaAdministrativaUsuarioService.java
src/main/java/com/servify/administracion/application/service/ObtenerMedidasAdministrativasDeUsuarioService.java
src/main/java/com/servify/administracion/application/service/ActualizarConfiguracionGeneralService.java
src/main/java/com/servify/administracion/application/service/ObtenerConfiguracionGeneralService.java
src/main/java/com/servify/administracion/application/service/ModerarPublicacionService.java
```

12. Recomendaciones para continuar

Orden sugerido:

1. Resolver la clase principal de Spring Boot o ajustar `ServifyApplicationTests`.
2. Implementar adaptadores de infraestructura para los repository ports propios.
3. Implementar adaptadores cross-module:
 - `UsuarioAutenticablePort`
 - `UsuarioAdministrablePort`
 - `PublicacionModerablePort`
 - `PublicacionesCompatiblesPort`
 - `ConfiguracionDistribucionPort`
4. Reemplazar gradualmente los `UnsupportedOperationException` por logica real.
5. Agregar controllers REST cuando la application layer este estable.
6. Agregar tests unitarios por service, mockeando puertos.
7. Agregar tests de integracion cuando exista infraestructura JPA.

13. Prompt sugerido para continuar con otro ChatGPT

Estoy trabajando en el backend de Servify, una app movil con backend Java 21 + Spring Boot + Maven, monolito modular en capas y API REST JSON. La arquitectura exige bajo acoplamiento, puertos in/out como interfaces, services como clases que implementan use cases, DTOs simples sin record, y comunicacion entre modulos mediante UUID y puertos cross-module. No se deben exponer entidades de dominio entre modulos.

Ya se agregaron esqueletos de clases faltantes para los modulos solicitudes, autenticacion y administracion. No hay logica real todavia: los metodos pendientes tienen TODO y UnsupportedOperationException.

Puntos importantes:

- solicitudes agrego results/commands para asignacion y contraofertas, ports ConfirmarAsignacionSolicitudUseCase, ResolverContraofertaUseCase, ObtenerEstadoAsignacionSolicitudUseCase, PublicacionesCompatiblesPort y ConfiguracionDistribucionPort, mas sus services.
- autenticacion agrego commands/results de credenciales y sesion, ports de login/token/password/refresh token/usuario autenticable, services correspondientes y el modelo RefreshToken.
- administracion agrego commands/results para medidas administrativas, configuracion general y moderacion de publicaciones, ports para usuarios administrables y publicaciones moderables, mas services.
- Los results grandes usan builder.
- ModerarPublicacionCommand usa String estadoDestino para no importar EstadoPublicacion desde publicaciones.
- No se debe duplicar gestion de categorias en administracion.
- mvn compile pasa correctamente.
- mvn test falla por falta de @SpringBootTestConfiguration o @SpringBootTestApplication detectable desde SerfivyApplicationTests, problema no relacionado con las clases nuevas.

Necesito continuar respetando esta arquitectura. Antes de implementar, revisar documentos tecnicos y clases existentes del repo, y mantener los limites de modulo.